23CSE202 DATABASE MANAGEMENT SYSTEMS
S3 B.Tech CSE
Department of Computer Science and Engineering
Amrita School of Computing, Amritapuri Campus

PROJECT REPORT

Group No : C8
Team Members:

Roll No	Student Name
AM.SC.U4CSE25218	GORRELA TULASI LASYA
AM.SC.U4CSE25220	HANSIKA L CHAWLA
AM.SC.U4CSE25234	MUMMADI MANJUNADHA REDDY
AM.SC.U4CSE25236	NANDANA JAYAKUMAR

Project Title : CineHall - An Online Movie Ticket Booking System

CONTENTS

1. Abstract 2
2. Introduction 3
3. Requirements (Functional & Non-functional) 4
4. UI Design 6
5. Database Design 9
6. Database Selection & Justification 18
7. Implementation 19
8. Testing & Validation 27
9. Results & Screenshots 29
10. Challenges Faced & Solutions 35
11. Conclusion & Future Enhancements 36
References 37

1. Abstract

The Online Movie Ticket Booking System tackles the common frustration of booking apps that freeze or double-book a seat during a blockbuster release, by designing a clean, normalized relational database behind the scenes. The goal is to link theatres, screens, movies, showtimes, seats and customers, track seats in real time so double bookings are eliminated through concurrency control, handle secure transactions and cancellations, and give admins clear insight into revenue and audience trends. The proposed system uses the relational model with relations for Theatre, Movie, Screen, Seat, Customer, Showtime, Price, Booking, Cancellation and Payment, and it serves three groups of users: customers who browse and book tickets, theatre administrators who manage schedules, screen layouts and pricing, and system administrators who oversee the platform, manage accounts and pull analytical reports. Its main functions include secure registration and login, a movie catalogue searchable by genre, date or theatre, an interactive seat layout that produces a digital ticket booking, and admin tools to add or update showtimes, adjust prices by seat type, process cancellations and view summaries of total tickets sold and revenue per movie.

2. Introduction

2.1 Problem Domain and Background

The entertainment and cinema industry handles thousands of customer transactions every day. A single multiplex theatre may run dozens of shows across multiple screens simultaneously, with hundreds of seats available for each. Managing this at scale, tracking which seats are available, processing payments, issuing tickets and handling cancellations requires a system that is fast, accurate and consistent always.

Movie ticket booking is a classic example of a high-concurrency data management problem. When a popular film releases, multiple customers attempt to book the same seats at the same moment. The system must resolve these conflicts instantly and correctly, ensuring that no seat is ever sold twice. Beyond the booking itself, the system must also maintain a consistent record of payments, handle cancellations with proper refund tracking and give theatre managers a clear, real-time picture of revenue and occupancy.

This is not just a software problem; it is fundamentally a data management problem. The quality of the underlying database design directly determines whether the system can handle real-world load without errors, inconsistencies or data loss.

2.2 Existing Scenario and Its Limitations

Many small and mid-sized theatres continue to rely on manual ticketing methods such as counter sales, phone bookings and paper records, or on simple digital systems that are not backed by a properly structured database. These approaches suffer from several serious limitations.

The most critical is the absence of concurrency control. When two staff members or two users attempt to book the same seat at the same time, there is no mechanism to prevent both from succeeding. This leads to double-bookings, which damage customer trust and requires manual intervention to resolve.

Data stored in spreadsheets or on the client-side using browser storage such as local Storage has no referential integrity. There are no enforced relationships between movies, showtimes, screens and seats, which means records can become orphaned or inconsistent; for example, a showtime may reference a screen that no longer exists, or a booking may have no corresponding payment record.

Manual and client-side systems also provide no audit trail. When a booking is cancelled, the record is typically deleted or overwritten, making it impossible to track refund status or dispute a transaction after the fact. Theatre managers have no reliable way to query total revenue, seats sold per show or booking trends over time.

Finally, such systems do not scale. As the number of movies, screens and customers grows, the lack of a structured, indexed database means queries become slow, reports become inaccurate and the system becomes difficult to maintain.

2.3 The Proposed System

CineHall is an online movie ticket booking system designed to address all the limitations described above through a normalized relational database built in PostgreSQL.

The system is designed for three groups of users. Customers can browse a movie catalogue categorized as Now Showing or Coming Soon, filter by genre and language, view available showtimes across multiple theatres and screens, and select seats on a live interactive seat map that distinguishes between Silver, Gold and Premium pricing tiers. On confirming their selection, they complete a payment and receive a unique ticket code. They can view their full booking history and cancel any upcoming booking from their account.

Theatre administrators access a separate console where they can add, edit and delete movies and showtimes, configure per-tier pricing for each show and search or cancel any customer booking. A dashboard displays total movies listed, active showtimes, cumulative tickets sold and total revenue from confirmed bookings only.

The database that underpins CineHall consists of ten tables: Theatre, Screen, Movie, Showtime, Seat, Customer, Booking, Booking_Seat, Payment and Cancellation, designed up to Boyce-Codd Normal Form. Primary keys, foreign keys, and UNIQUE, CHECK and NOT NULL constraints enforce data integrity at the database level itself, independent of application logic. A Node.js and Express server exposes a REST API that connects the HTML, CSS and JavaScript front end to the database, keeping all business logic and price calculations on the server rather than the client.

Every booking run inside a database transaction with row-level locking, so two customers attempting to book the same seat simultaneously will never both succeed, one will be told the seat was just taken and prompted to choose another. Every cancellation writes a record to the Cancellation table rather than deleting the booking, preserving a complete and permanent audit history. The system was verified against 29 functional and constraint test cases, all of which passed.

3. Requirements (Functional & Non-functional)

3.1 Functional requirements

ID	Requirement
FR1	**Authentication:** customers can register with name, email, phone and password and later sign in. Administrators sign in with a separate admin login.
FR2	**Browse movies:** list movies as Now Showing or Coming Soon with genre, language, duration, certificate and rating.
FR3	**Search showtimes:** view the dates, cinemas and times available for a movie.
FR4	**Seat selection:** show a live seat map of the chosen show, marking booked seats and the Silver, Gold and Premium tiers.
FR5	**Booking:** reserve up to 8 seats, record the payment and generate a unique ticket code.
FR6	**Booking management:** a customer can view their bookings and cancel a booking before the show date.
FR7	**Movie management (admin):** insert, update and delete movies.
FR8	**Showtime management (admin):** schedule a show on a particular screen with date, time and tier prices, and delete showtimes.
FR9	**Booking monitoring (admin):** search all bookings by name, email or ticket code and cancel any booking.
FR10	**Reports (admin):** dashboard with number of movies, showtimes, tickets sold and total revenue.

3.2 Non-functional requirements

Quality	Requirement
Data integrity	All rules are enforced in the database with primary keys, foreign keys and UNIQUE, CHECK and NOT NULL constraints.
Reliability	Bookings are saved with transactions (BEGIN, COMMIT, ROLLBACK), so a failure never leaves a half-saved booking.
Concurrency	Two users must not be able to book the same seat; the show row is locked while a booking is processed.
Security	Passwords are stored as salted scrypt hashes; all SQL uses parameterized queries; admin actions require an admin token; customers see only their own bookings; prices are calculated on the server.

Quality	Requirement
Usability	Simple pages that a non-technical user can follow: pick a movie, pick seats, pay, get a ticket.
Responsiveness	The layout adapts from desktop screens to small mobile screens.
Portability	Runs in any modern browser; the server runs on Windows, Linux or macOS with Node.js and PostgreSQL.
Scalability	The normalized schema and indexes allow more theatres, screens and shows to be added without redesign.

3.3 Business rules and implementation status

The Database Design defined six business rules. The table shows how each one is handled in the final system.

Business rule	Status in the final system
Unique account (email)	**Implemented.** email is UNIQUE in Customer; a duplicate sign-up returns an error (test T2, T24).
No overlapping showtimes on a screen	**Partly implemented.** A UNIQUE constraint on (screen, date, time) stops two shows starting at the same time on one screen. Checking movie duration plus a cleaning buffer is future work.
Double-booking prevention	**Implemented.** Transaction with row lock and seat check (tests T7, T8).
Pricing by seat type	**Implemented.** Each showtime stores Silver, Gold and Premium prices; the server computes the total (test T7).
10-minute payment deadline	**Not implemented.** Payment is a demo and is confirmed immediately. Listed as future work.
Cancellation window and refunds	**Partly implemented.** A customer can cancel until the show date; seats are released at once and a Cancellation row with refund status Pending is created. A 2-hour-before-start rule is future work.
Maximum 8 seats per booking	**Implemented.** Frontend enforces a cap of 8 seats per transaction with a warning toast.
No cancellation after show date	**Implemented.** Server rejects cancellation requests for shows that have already passed (test T-cancel).
Server-side price calculation	**Implemented.** Total amount is always computed on the server from seat types and showtime prices; the client cannot override it.

4. UI Design

CineHall has two distinct interfaces: a customer-facing website and an administrator console both built using plain HTML, CSS and JavaScript and served through the Express server. The

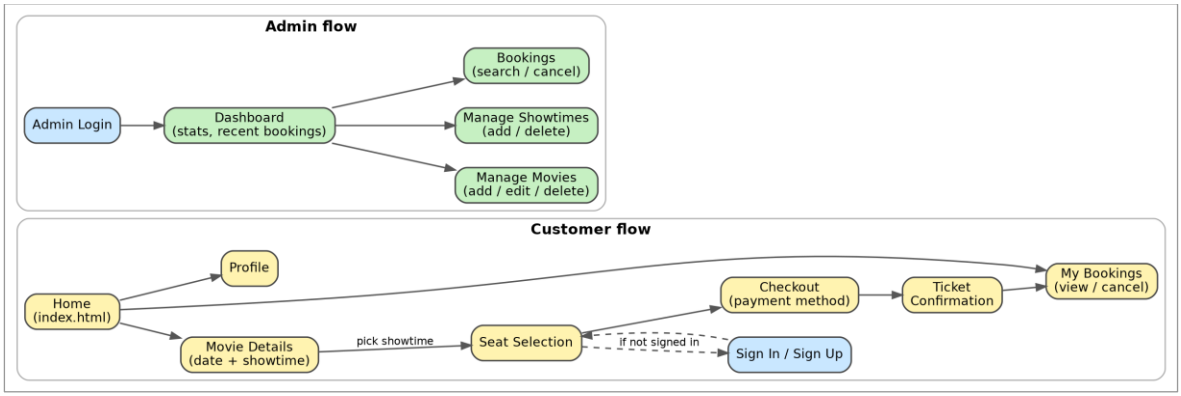
visual design follows a dark cinema-hall theme with gold accents for primary actions and a contrasting tone for destructive ones. All pages share a single stylesheet and a common JavaScript layer, keeping the interface consistent across every screen.

4.1 Design System and Navigation Flow

The interface uses a carefully chosen fonts, a bold display font for headings, a clean sans-serif for body text and a monospace font for ticket codes, prices and labels. A circular gradient logo mark sits beside the CineHall wordmark in the top navigation bar on every customer page.

Customer pages have a sticky top navigation bar that shows the logo, links to key sections of the site and either a Sign In button or the logged-in customer's name with a Logout option. On smaller screens the navigation collapses into a mobile-friendly menu to maintain usability. Admin pages replace the top bar with a sidebar that links to the main admin sections and includes a logout button at the bottom. The sidebar similarly adapts to smaller screens by collapsing into a slide-in drawer.

The customer navigation flow is linear by design:
Home → Movie Details → Seat Selection → Checkout → Confirmation
Authentication is required from the seat selection step onward. If a customer tries to reach a protected page without being signed in, they are redirected to the login page and returned to their original destination after signing in. The admin flow is independent — all admin pages verify an active admin session and redirect to the admin login page if none is found.



Navigation flow of the customer and admin sides

4.2 Customer Screens

Home Page (index.html)

The home page is the entry point for all customers. It opens with a full-width hero banner that sets the tone of the application. Below it, a horizontally scrolling row displays all currently showing movies as poster cards, each showing the movie title, genre, rating and a visual identifier. A separate section below lists upcoming movies. Clicking any currently showing poster takes the customer to that movie's details page.

Login and Sign Up (login.html)
The login page presents two tabbed forms on a centred card: one for returning customers and one for new registrations. The sign-in form asks for the customer's email and password. The registration form collects the information needed to create a new account. Submitting either form creates a session and redirects the customer to wherever they were trying to go before, or to their profile page by default.

Movie Details (movie-details.html)
This page shows the full details of a selected movie including its title, genre, language, duration, certificate and synopsis alongside a visual poster block. For upcoming movies, a message is shown in place of the booking options. For currently showing movies, the customer can browse available dates and then select a showtime from the cinemas offering that film. Selecting a showtime takes the customer to the seat selection page for that specific show.

Seat Selection (seats.html)
This is the most interactive screen in the system. A visual representation of the screen is shown at the top to orient the customer. Below it, the full seat layout for the selected showtime is rendered as a grid. Seats are grouped into pricing tiers, visually distinguished by color, so the customer can easily identify the type and price of each seat. Seats already taken by other customers are shown as disabled and cannot be selected. Clicking an available seat toggles its selection and highlights it. A fixed booking bar at the bottom of the screen shows the number of seats selected, their labels, the running total and a button to confirm the selection — which remains disabled until at least one seat is chosen. If the customer attempts to select more seats than the allowed limit, a warning notification is shown.

Checkout (checkout.html)
The checkout page presents an order summary showing the movie, cinema, date, time, selected seats and total amount. Below the summary, a form prefilled with the customer's details from their session asks them to confirm their information and choose a payment method. Confirming the order completes the booking, saves all records to the database and takes the customer to the confirmation page.

Booking Confirmation (confirmation.html)
The confirmation page renders a stylized digital ticket. The upper portion shows the cinema, movie, date, time and seat details. A visual divider separates it from the lower portion, which displays the unique ticket code, the customer's name and the amount paid, alongside a decorative placeholder for a QR code. Two buttons allow the customer to view all their bookings or return to the home page to book another movie.

My Bookings (my-bookings.html)
This page lists all bookings associated with the signed-in customer's account. Each booking is shown as a card with the movie, show details, selected seats, a status badge indicating whether the booking is confirmed or cancelled, and the ticket code. A cancel option is available for

eligible bookings. If a booking cannot be cancelled — because it has already been cancelled or the show has passed — the option is disabled with a visual indication of why.

Profile (profile.html)
The profile page shows a visual avatar built from the customer's initials alongside their name, email and account type. Buttons link to the customer's booking history and allow them to log out.

4.3 Administrator Screens

Admin Login (admin-login.html)
The admin login page is separate from the customer login. It presents the CineHall logo with an Admin Console label and a form for administrator credentials. Invalid credentials are rejected by the server. On successful login, an admin session is created, and the administrator is taken to the dashboard.

Dashboard (admin-dashboard.html)
The dashboard is the first screen an administrator sees after logging in. A row of summary cards at the top gives an at-a-glance view of the system: total movies listed, active showtimes, tickets sold and revenue from confirmed bookings. Below the cards, a table shows the most recent bookings across all customers with key details briefly.

Movies (admin-movies.html)
The movies page presents a table of all movies in the system. Each row shows the key details of a movie and provides options to edit or delete it. Clicking edit opens a modal form prefilled with that movie's current details, which the administrator can update and save. Clicking add opens the same form empty. Deleting a movie shows a confirmation dialog that warns the administrator about the cascading effect on associated showtimes and bookings before proceeding.

Showtimes (admin-shows.html)
The showtimes page lists all scheduled shows with their movie, cinema, date, time, pricing and current booking count. Each showtime can be removed using a delete button. The Add Showtime form lets the administrator select a movie, specify the cinema, date, time and pricing for each seat tier before saving the new showtime to the database.

Bookings (admin-bookings.html)
The admin bookings page shows all bookings across every customer in a searchable table. The administrator can search by customer name, email or ticket code to quickly locate a specific booking. Each row shows the full booking details along with its status. Confirmed bookings can be cancelled directly from this page, which updates the status and records the cancellation.

4.4 Responsiveness

Every page is fully responsive and adapts to any screen size, from large desktop monitors down to small mobile devices. The layout uses flexible units and modern CSS layout techniques throughout, so no content overflows or becomes inaccessible on smaller screens. Wide elements such as the seat grid, booking tables and admin tables scroll horizontally within their containers rather than breaking the page layout. All interactive elements: buttons, seat cells, navigation links have clear hover and focus states to indicate interactivity.

5. Database Design

5.1 Entities and attributes

The ER model has nine entities. The many-to-many link between Booking and Seat appears as the relationship **Reserves** and becomes its own table, **Booking_Seat**, when the model is converted to tables, which gives ten tables in total. Primary keys are marked (PK).

Entity	Description	Attributes
User	Represents any registered individual in the system.	UserID (Primary Key), Name, Email, Phone, PasswordHash, Role (Customer, Theater Admin, System Admin).
Movie	Represents films available for screening.	MovieID (Primary Key), Title, Genre, Duration (in minutes), Language, ReleaseDate, Rating.
Theater	Represents the physical cinema locations hosting screenings.	TheaterID (Primary Key), Name, City, Address, TotalScreens.
Screen	Represents individual auditoriums within a theater.	ScreenID (Primary Key), ScreenNumber, TotalSeats, ScreenType
Seat	Represents physical seating units inside a screen.	SeatID (Primary Key), SeatNumber, RowLetter, SeatType
Showtime	Represents a specific screening of a movie on a screen at a scheduled time.	ShowtimeID (Primary Key), ShowDate, StartTime, EndTime, BasePrice.
Booking	Represents a ticket reservation transaction initiated by a customer.	BookingID (Primary Key), BookingDate, TotalAmount, Status
Ticket	Represents individual seats reserved within a booking.	TicketID (Primary Key), SeatPrice, TicketStatus
Payment	Represents the transaction processing for a booking.	PaymentID (Primary Key), PaymentDate, Amount, PaymentMethod,PaymentStatus

5.2 Relationships and cardinalities

Relationship	Entities	Cardinality	Meaning
Has	Theatre – Screen	1 : N	A theatre has one or more screens; each screen belongs to one theatre.
Contains	Screen – Seat	1 : N	A screen contains many seats; each seat belongs to one screen.
Hosts	Screen – Showtime	1 : N	A screen hosts many showtimes; each showtime is on one screen.
Scheduled for	Movie – Showtime	1 : N	A movie can have many showtimes; each showtime is for one movie.
Booked for	Showtime – Booking	1 : N	A showtime can have many bookings; each booking is for one showtime.
Makes	Customer – Booking	1 : N	A customer can make many bookings; each booking belongs to one customer.
Reserves	Booking – Seat	M : N	A booking reserves one or more seats; a seat can appear in many bookings over time (different shows).
Paid by	Booking – Payment	1 : N	A booking has a payment record (more than one if a payment attempt fails and is retried).
Cancelled as	Booking – Cancellation	1 : 1	A booking may have at most one cancellation record.

5.3 ER diagram

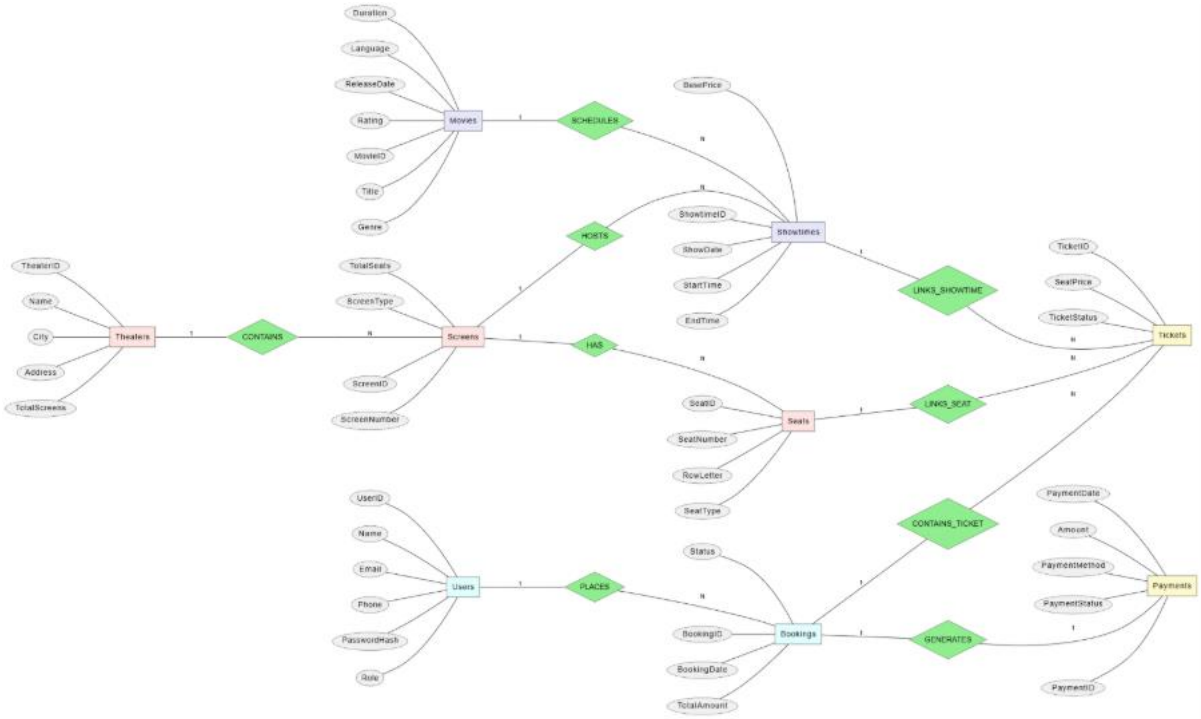


Figure 2: Entity Relationship (Chen) diagram of CineHall: rectangles are entities, ovals are attributes (key attributes underlined), diamonds are relationships

5.4 Conversion of the ER model into relational tables

- **Each entity becomes a table** and its key attribute becomes the primary key (a SERIAL integer, which PostgreSQL fills automatically).
- **Each 1 : N relationship** is implemented by placing the primary key of the "1" side as a foreign key in the table of the "N" side: Screen.theatre_id, Seat.screen_id, Showtime.movie_id, Showtime.screen_id, Booking.customer_id, Booking.showtime_id, Payment.booking_id.
- **The M : N relationship Reserves** (Booking – Seat) becomes the junction table **Booking_Seat** with foreign keys booking_id and seat_id.
- **The 1 : 1 relationship** between Booking and Cancellation is implemented with a foreign key booking_id in Cancellation; the application creates only one cancellation record per booking.
- **Ticket price and total:** the three tier prices are stored in Showtime; the total of a booking is stored in Booking.total_amount at the moment of booking.
- **Foreign keys use ON DELETE CASCADE**, so deleting a parent row (for example a movie) also removes its dependent rows.

5.5 Relational schema

PK = primary key, FK = foreign key. Every column listed is NOT NULL unless stated otherwise.

1. Customer Table

Column Name	Data Type	Constraints	Description
customer_id	SERIAL	**PK**	Unique identifier for the customer
name	VARCHAR(100)	NOT NULL	Customer's full name
email	VARCHAR(100)	UNIQUE, NOT NULL	Unique login email
phone	VARCHAR(15)	UNIQUE, NOT NULL	Contact phone number
password	VARCHAR(255)	NOT NULL	Secure password

2. Theatre Table

Column Name	Data Type	Constraints	Description
theatre_id	SERIAL	**PK**	Unique identifier for the theatre
name	VARCHAR(100)	NOT NULL	Theatre name
location	VARCHAR(255)	NOT NULL	Address location
contact_info	VARCHAR(50)	NOT NULL	Support contact info
total_screens	INT	NOT NULL	Number of screens

3. Screen Table

Column Name	Data Type	Constraints	Description
screen_id	SERIAL	**PK**	Unique identifier for the screen
theatre_id	INT	**FK (Theatre)**	Parent theatre reference
screen_number	INT	NOT NULL	Screen number
seating_capacity	INT	NOT NULL	Total seats available

4. Seat Table

Column Name	Data Type	Constraints	Description
seat_id	SERIAL	**PK**	Unique identifier for the seat
screen_id	INT	**FK (Screen)**	Parent screen reference
seat_row	VARCHAR(2)	NOT NULL	Row identifier (e.g., A, B)
seat_number	INT	NOT NULL	Seat number in row
seat_type	VARCHAR(20)	NOT NULL	Tier (Silver, Gold, Premium)

5. Movie Table

Column Name	Data Type	Constraints	Description
movie_id	SERIAL	**PK**	Unique identifier for the movie
title	VARCHAR(150)	NOT NULL	Movie title
genre	VARCHAR(50)	NOT NULL	Movie genre
language	VARCHAR(50)	NOT NULL	Language
duration	INT	NOT NULL	Duration in minutes
certificate	VARCHAR(10)	NOT NULL	Censor rating (U, UA, A)
status	VARCHAR(20)	NOT NULL	Status (Now Showing / Coming Soon)
synopsis	TEXT		Plot summary
rating	DECIMAL(3,1)		Average rating

6. Showtime Table

Column Name	Data Type	Constraints	Description
showtime_id	SERIAL	**PK**	Unique identifier for the showtime
movie_id	INT	**FK (Movie)**	Movie reference
screen_id	INT	**FK (Screen)**	Screen reference
show_date	DATE	NOT NULL	Screening date
show_time	TIME	NOT NULL	Screening time
silver_price	DECIMAL(8,2)	NOT NULL	Price for Silver seats
gold_price	DECIMAL(8,2)	NOT NULL	Price for Gold seats
premium_price	DECIMAL(8,2)	NOT NULL	Price for Premium seats

7. Booking Table

Column Name	Data Type	Constraints	Description

booking_id	SERIAL	**PK**	Unique identifier for the booking
customer_id	INT	**FK (Customer)**	Customer reference
showtime_id	INT	**FK (Showtime)**	Showtime reference
ticket_code	VARCHAR(20)	UNIQUE, NOT NULL	Generated unique ticket string
booking_time	TIMESTAMP	NOT NULL	Timestamp of booking
total_amount	DECIMAL(8,2)	NOT NULL	Total cost
status	VARCHAR(20)	NOT NULL	Status (Confirmed, Cancelled)

8. Booking_Seat Table

Column Name	**Data Type**	**Constraints**	**Description**
booking_seat_id	SERIAL	**PK**	Unique identifier
booking_id	INT	**FK (Booking)**	Booking reference
seat_id	INT	**FK (Seat)**	Seat reference

9. Payment Table

Column Name	**Data Type**	**Constraints**	**Description**
payment_id	SERIAL	**PK**	Unique identifier for payment
booking_id	INT	**FK (Booking)**	Booking reference
amount	DECIMAL(8,2)	NOT NULL	Transaction amount
payment_method	VARCHAR(30)	NOT NULL	Method (UPI, Card, etc.)
payment_status	VARCHAR(20)	NOT NULL	Status (Success, Failed)
payment_time	TIMESTAMP	NOT NULL	Timestamp

10. Cancellation Table

Column Name	Data Type	Constraints	Description
cancellation_id	SERIAL	**PK**	Unique identifier for cancellation
booking_id	INT	**FK (Booking)**	Booking reference
refund_status	VARCHAR(20)	NOT NULL	Refund status
reason	TEXT		Reason for cancellation
cancelled_at	TIMESTAMP	NOT NULL	Timestamp of cancellation

5.6 Functional dependencies

	Function Dependency	Significance
FD1	Customer_id-> customer_name, email , phone, password	Ensures a customer's personal details are uniquely tied to their ID, centralizing profile management and preventing redundant data entry across multiple bookings.
FD2	Booking_seat_id->booking_id,seat_id	Resolves the many-to-many relationship between bookings and seats by mapping a specific reserved ticket strictly to its parent transaction and physical seat.
FD3	booking_id -> booking_time, total_amount, booking_status, customer_id, showtime_id, ticket_code	Centralizes all transactional details of a booking to its unique ID, ensuring transaction integrity and establishing the core link between the customer and the showtime.
FD4	showtime_id-> show_date, show_time, silver_price, gold_price, premium_price	Guarantees that a specific showtime dictates exactly when and where a movie plays, along with its dynamic pricing tiers, ensuring scheduling consistency.
FD5	movie_id-> title, genre, language, duration, certificate, movie_status, synopsis, rating	Isolates static movie data to a single identifier, meaning movie details are stored exactly once and referenced by ID, preventing update anomalies.
FD6	screen_id -> screen_number, seating_capacity	Links a specific physical screen to its capacity and parent theatre, ensuring accurate seating limits

		and location routing for the booking system.
FD7	Theatre_id -> theatre_name, location, contact_info, total_screens	Centralizes theatre information, preventing the duplication of addresses and contact info across multiple screens and showtimes.
FD8	Seat_id -> seat_row, seat_number, seat_type	Defines the physical properties and tier of a single seat within a specific screen, serving as the basis for double-booking prevention and dynamic pricing.
FD9	Payment_id -> payment_amount, payment_method, payment_status, booking_id	Ties payment gateway data to a specific transaction ID, ensuring accurate financial tracking independent of the booking itself.
FD10	Cancellation_id ->booking_id, refund_status, reason, cancelled_at	Isolates cancellation data to a specific event, keeping the main booking table clean while tracking refund states and chronological timestamps.
FD11	Email -> customer_id	Since every email is unique, it can serve as candidate key
FD12	ticket_code ->booking_id, customer_id, showtime_id, booking_time, total_amount, status	Guarantees that a generated ticket string can uniquely reverse-lookup the entire booking transaction, acting as a candidate key.

5.7 Normalization

To show the normalization, we start from one large unnormalized (UNF) relation that holds all the information of a booking in one place:

```
CineHall(booking_id, customer_id, theatre_id, screen_id, movie_id,
showtime_id, seat_id, payment_id, cancellation_id ,customer_name,
email, phone, password, theatre_name, location, contact_info,
total_screens, screen_number, seating_capacity, title, genre,
language, duration, certificate, movie_status, synopsis, rating,
show_date, show_time, silver_price, gold_price, premium_price,
ticket_code, booking_time, total_amount, booking_status,{seat_row,
seat_number, seat_type},  {payment_amount, payment_method,
payment_status, payment_time}, {refund_status, cancellation_reason,
cancelled_at} )
```

This structure repeats the same customer, theatre, movie and seat information in every booking record, which causes redundancy and update, insertion and deletion anomalies. The candidate key of the booking-level record is booking_id: using FD7 and, by transitivity, FD1, FD6, FD5,

FD3 and FD2, the closure $\{booking_id\}^+$ contains every attribute of the booking record, and since the key has a single attribute it is minimal.

Normal form	What was done	Result
1NF	The repeating groups {seat_row, seat_number, seat_type}, {payment details} and {cancellation details} were removed into their own relations: SEAT, BOOKING_SEAT, PAYMENT and CANCELLATION. All attributes are now atomic.	No repeating groups.
2NF	Every relation has a single-attribute primary key (customer_id, theatre_id, screen_id, seat_id, movie_id, showtime_id, booking_id, booking_seat_id, payment_id, cancellation_id). With a one-column key, a partial dependency is impossible.	No partial dependencies.
3NF	Transitive dependencies such as booking_id → customer_id → customer details, booking_id → showtime_id → movie_id → movie details and booking_id → showtime_id → screen_id → theatre_id → theatre details were removed by creating CUSTOMER, THEATRE, SCREEN, MOVIE and SHOWTIME.	No non-key attribute depends on another non-key attribute.
BCNF	In every relation the determinant of every non-trivial dependency (FD1–FD10) is the primary key, which is a superkey. The extra candidate keys email (Customer) and ticket_code (Booking) are also superkeys.	Every determinant is a superkey.
4NF	There are no non-trivial multivalued dependencies with a non-superkey determinant, because the independent multi-valued facts (seats, payments, cancellations) are already in separate relations.	No multivalued dependencies.
5NF	No non-trivial join dependency exists that is not implied by the candidate keys; the relations are already joined only through primary and foreign keys.	No further decomposition needed.

The final decomposition is the ten relations CUSTOMER, THEATRE, SCREEN, SEAT, MOVIE, SHOWTIME, BOOKING, BOOKING_SEAT, PAYMENT and CANCELLATION described in Section 5.5.

6. Database Selection & Justification

The project uses **PostgreSQL**, an open-source object-relational database system, together with the `pg` (node-postgres) driver for Node.js.

6.1 Why PostgreSQL is suitable

- **ACID transactions:** a booking inserts into Booking, Booking_Seat and Payment. PostgreSQL commits all three or none, which keeps the data consistent.
- **Row-level locking:** `SELECT ... FOR UPDATE` locks the show while a booking is checked and saved, so two customers cannot take the same seat at the same moment.
- **Rich constraints:** PRIMARY KEY, FOREIGN KEY, UNIQUE (including composite keys), CHECK and NOT NULL let the database itself reject invalid data.
- **Powerful SQL:** joins, subqueries, EXISTS, GROUP BY / HAVING, CASE, `generate_series` for sample data, and aggregate functions such as `string_agg` and `array_agg` are used throughout the project.
- **Data types:** SERIAL keys, DATE, TIME, TIMESTAMP and exact DECIMAL types of suit money and schedules.
- **Performance and scalability:** indexes, a query planner and the ability to handle many concurrent connections through a connection pool.
- **Security:** role-based access and support for encrypted connections.
- **Cost and support:** free and open source, with detailed official documentation and an active community; it runs on Windows, Linux and macOS and has tools such as pgAdmin.
- **Programming support:** PL/pgSQL for triggers and functions, which is useful for the future enhancements (for example automatic release of unpaid bookings).

6.2 Comparison with other options

Feature	PostgreSQL	MySQL	Oracle
Licence / cost	Free, open source	Community edition is free	Commercial licence for full features
ACID transactions	Yes	Yes (InnoDB engine)	Yes
Row-level locking with SELECT FOR UPDATE	Yes	Yes (InnoDB)	Yes
CHECK constraints	Yes	Yes (from version 8.0.16)	Yes
Procedural language	PL/pgSQL	Stored procedures	PL/SQL
Ease of setup for a student project	Easy	Easy	Heavier installation

All three could be used, but PostgreSQL combines strict standards-compliant SQL, strong transaction support and zero cost, which is the best fit for this project.

7. Implementation

7.1 Architecture and technologies

CineHall follows a three-tier architecture. The browser (presentation tier) calls a REST API written with Express (application tier), which runs parameterized SQL on PostgreSQL (data tier). The browser never talks to the database directly.

Tier	Technology	Role
Presentation	HTML, CSS, JavaScript (`fetch` API)	Pages, seat map, forms, admin console
Application	Node.js with Express 5	REST API, validation, authentication, transactions
Data	PostgreSQL with the `pg` driver	Tables, constraints, queries, transactions

Project files: `server.js` (API), `schema.sql` (tables), `seed.sql` (sample data), `package.json`, and the `public/` folder with all pages, `script.js`, `admin-shell.js` and `styles.css`. The source code is available at https://github.com/Hansi-111/movie-ticket-booking-system-ui.

7.2 Database creation and table creation

```sql
CREATE TABLE Theatre (
    theatre_id SERIAL PRIMARY KEY,
    name VARCHAR(100) NOT NULL,
    location VARCHAR(255) NOT NULL,
    contact_info VARCHAR(50) NOT NULL,
    total_screens INT NOT NULL
);

CREATE TABLE Screen (
    screen_id SERIAL PRIMARY KEY,
    theatre_id INT NOT NULL REFERENCES Theatre(theatre_id) ON DELETE CASCADE,
    screen_number INT NOT NULL,
    seating_capacity INT NOT NULL,
    UNIQUE (theatre_id, screen_number)
);

CREATE TABLE Movie (
    movie_id SERIAL PRIMARY KEY,
    title VARCHAR(150) NOT NULL,
    genre VARCHAR(50) NOT NULL,
    language VARCHAR(50) NOT NULL,
    duration INT NOT NULL CHECK (duration > 0),
    certificate VARCHAR(10) NOT NULL,
    status VARCHAR(20) NOT NULL CHECK (status IN ('Now Showing','Coming Soon')),
    synopsis TEXT,
    rating DECIMAL(3,1) CHECK (rating BETWEEN 0 AND 10)
);
```

The complete DDL for all ten tables is in `schema.sql`.

7.3 Sample data

`seed.sql` inserts 2 theatres, 3 screens, 6 movies and showtimes created relative to today's date. The 240 seats (8 rows A–H × 10 seats per screen; rows A–C Silver, D–F Gold, G–H Premium) are generated with one INSERT using `generate_series`:

```
INSERT INTO Theatre (name, location, contact_info, total_screens) VALUES
('CineHall Downtown', 'MG Road, Kochi', '0484-1234567', 2),
('CineHall Mall', 'Lulu Mall, Edappally', '0484-7654321', 1);

INSERT INTO Screen (theatre_id, screen_number, seating_capacity) VALUES
(1,1,80),(1,2,80),(2,1,80);

-- -- 8 rows (A-H) x 10 seats per screen: A-C Silver, D-F Gold, G-H Premium --
INSERT INTO Seat (screen_id, seat_row, seat_number, seat_type)
SELECT sc.screen_id, chr(64 + r), n,
CASE WHEN r <= 3 THEN 'Silver' WHEN r <= 6 THEN 'Gold' ELSE 'Premium' END
FROM Screen sc, generate_series(1,8) r, generate_series(1,10) n;

INSERT INTO Movie (title, genre, language, duration, certificate, status, synopsis, rating) VA
('Starfall Odyssey','Sci-Fi','English',148,'UA','Now Showing','A crew races to reach a dying s
('The Last Monsoon','Drama','Malayalam',132,'U','Now Showing','A family reunites at their ance
('Iron Verdict','Action','Hindi',141,'UA','Now Showing','A disgraced cop has 24 hours to clear
('Whispers in Glass','Thriller','English',119,'A','Now Showing','A glassblower realises someon
('Realm of Embers','Fantasy','English',155,'UA','Coming Soon','Three kingdoms, one dragon egg,
('Crown of Dust','Historical','Hindi',162,'UA','Coming Soon','The rise and fall of a desert em

INSERT INTO Showtime (movie_id, screen_id, show_date, show_time, silver_price, gold_price, pre
SELECT m, s, CURRENT_DATE + d, t::time, 150, 220, 320
FROM (VALUES (1,1),(2,2),(3,3),(4,1)) AS mv(m,s),
generate_series(0,2) d,
(VALUES ('10:30'),('14:00'),('18:45'),('21:30')) AS tm(t)
WHERE (m + d + EXTRACT(HOUR FROM t::time)::int) % 2 = 0
ON CONFLICT DO NOTHING;
```

7.4 REST API and CRUD operations

Method and endpoint	SQL operation	Access
POST /api/auth/signup	INSERT into Customer (password hashed)	Public
POST /api/auth/login	SELECT from Customer by email	Public
GET /api/movies	SELECT from Movie	Public
POST /api/movies	INSERT into Movie	Admin
PUT /api/movies/:id	UPDATE Movie	Admin
DELETE /api/movies/:id	DELETE from Movie (cascades)	Admin
GET /api/shows	JOIN Showtime, Screen, Theatre	Public
GET /api/shows/:id/seats	SELECT seats with EXISTS check for booked seats	Public
POST /api/shows	INSERT into Showtime	Admin
DELETE /api/shows/:id	DELETE from Showtime	Admin
GET /api/bookings	JOIN of Booking, Customer, Showtime, Movie, Screen, Theatre, Seat	Customer (own) / Admin (all)
POST /api/bookings	Transaction: INSERT Booking, Booking_Seat, Payment	Customer
DELETE /api/bookings/:id	Transaction: UPDATE Booking, INSERT Cancellation	Customer (own) / Admin

7.5 Important SQL queries and their outputs

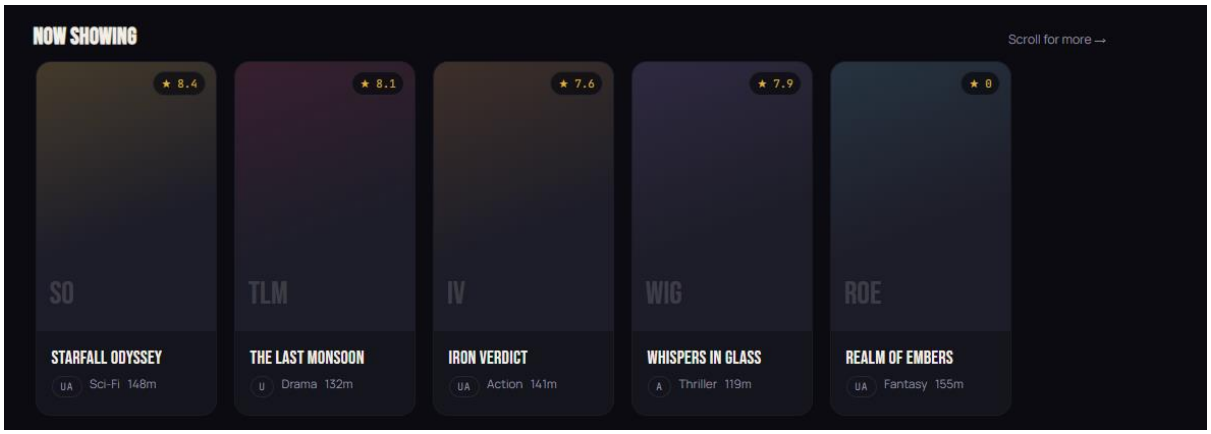
Q1. List all movies (SELECT)

Shows the movie catalogue used by the home page.

```
const MOVIE_SELECT = `
  SELECT movie_id AS id, title, genre, language AS lang, duration, certificate AS rating,
         CASE status WHEN 'Now Showing' THEN 'now' ELSE 'upcoming' END AS status,
         COALESCE(synopsis,'') AS synopsis, COALESCE(rating,0)::float AS score
  FROM Movie`;

app.get('/api/movies', async (req, res) => {
  const r = await pool.query(MOVIE_SELECT + ' ORDER BY movie_id');
  res.json(r.rows);
});
```

Output:

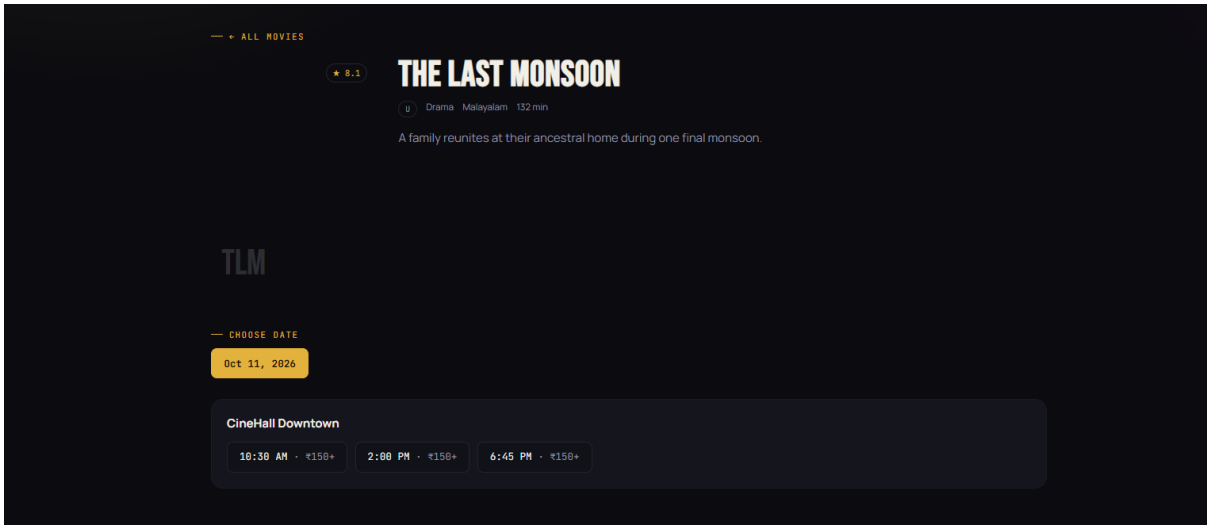


Q2. Showtimes with movie, theatre and prices (JOIN)

Joins four tables to list upcoming shows.

```
const SHOW_SELECT = `
  SELECT st.showtime_id AS id, st.movie_id AS "movieId", st.screen_id AS "screenId",
         t.name AS cinema, sc.screen_number AS "screenNumber",
         to_char(st.show_date, 'YYYY-MM-DD') AS date,
         to_char(st.show_time, 'FMHH12:MI AM') AS time,
         json_build_object('silver', st.silver_price::float, 'gold', st.gold_price::float,
                           'premium', st.premium_price::float) AS price,
         (SELECT COUNT(*)::int FROM Booking_Seat bs JOIN Booking b ON b.booking_id = bs.booking_id
           WHERE b.showtime_id = st.showtime_id AND b.status = 'Confirmed') AS "bookedCount"
  FROM Showtime st
  JOIN Screen sc ON sc.screen_id = st.screen_id
  JOIN Theatre t ON t.theatre_id = sc.theatre_id`;
```

Output:

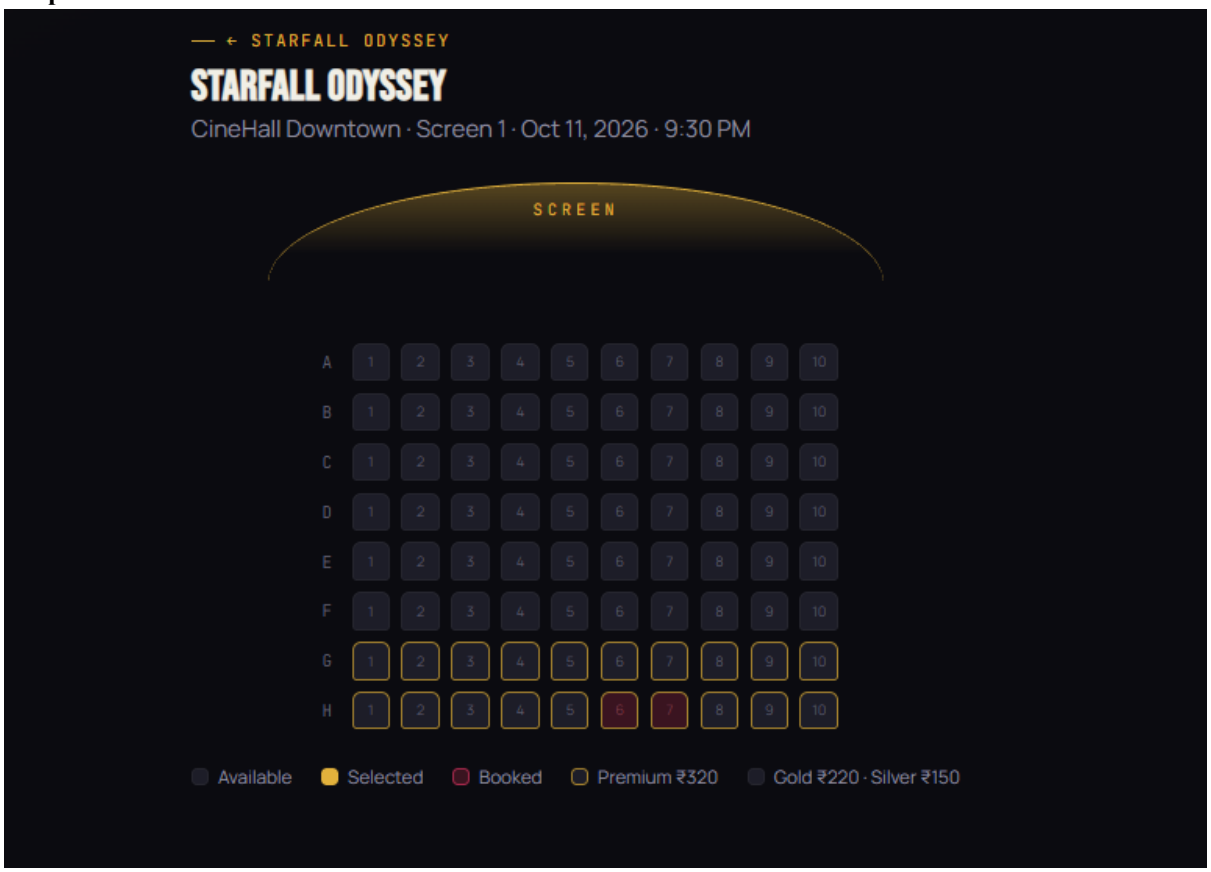


Q3. Seat availability per show (LEFT JOIN, GROUP BY, HAVING)

Counts booked and available seats for shows that have at least one confirmed seat. Cancelled bookings are ignored, so their seats count as available.

```
const r = await pool.query(
  `SELECT s.seat_id AS id, s.seat_row AS "row", s.seat_number AS number, s.seat_type AS type,
          EXISTS (SELECT 1 FROM Booking_Seat bs JOIN Booking b ON b.booking_id = bs.booking_id
                  WHERE b.showtime_id = $1 AND b.status = 'Confirmed' AND bs.seat_id = s.seat_id)
   FROM Seat s
   WHERE s.screen_id = (SELECT screen_id FROM Showtime WHERE showtime_id = $1)
   ORDER BY s.seat_row, s.seat_number`,
  [req.params.id]
```

Output:

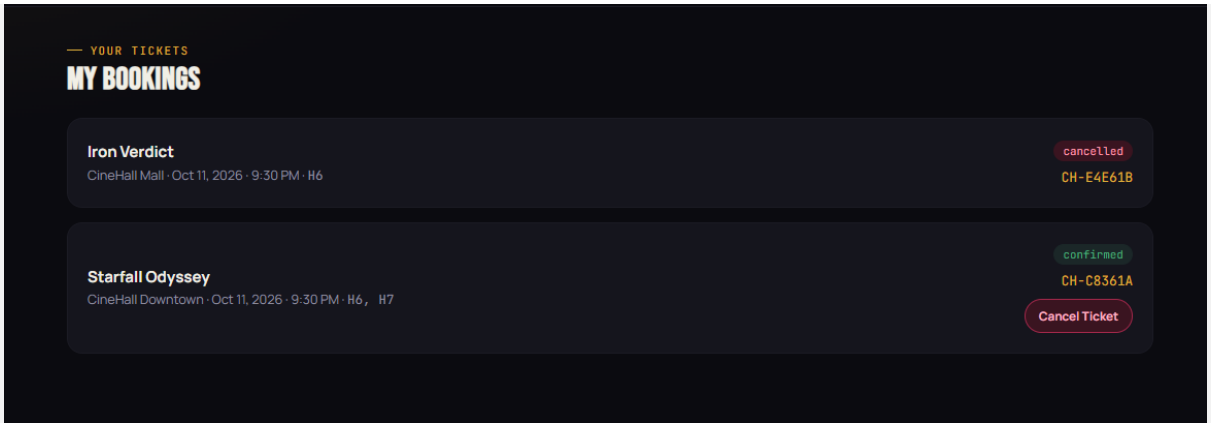


Q4. Bookings with customer, movie and seats (multi-table JOIN, string_agg)

Used by My Bookings and the admin Bookings page. The first booking was cancelled during testing.

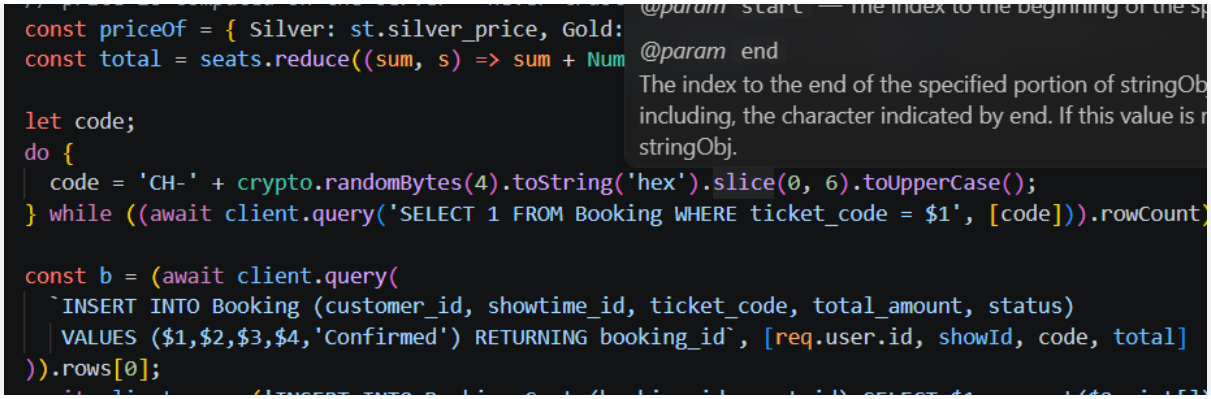
```
SELECT b.booking_id AS id, b.ticket_code AS code, b.showtime_id AS "showId", st.movie_id AS "movi
       m.title AS "movieTitle", t.name AS cinema,
       to_char(st.show_date, 'YYYY-MM-DD') AS date, to_char(st.show_time, 'FMHH12:MI AM') AS time
       COALESCE((SELECT array_agg(s.seat_row || s.seat_number ORDER BY s.seat_row, s.seat_number)
                 FROM Booking_Seat bs JOIN Seat s ON s.seat_id = bs.seat_id
                 WHERE bs.booking_id = b.booking_id), '{}') AS seats,
       b.total_amount::float AS total, c.name, c.email, c.phone,
       LOWER(b.status) AS status, b.booking_time AS "timestamp"
FROM Booking b
JOIN Customer c ON c.customer_id = b.customer_id
JOIN Showtime st ON st.showtime_id = b.showtime_id
JOIN Movie m ON m.movie_id = st.movie_id
JOIN Screen sc ON sc.screen_id = st.screen_id
JOIN Theatre t ON t.theatre_id = sc.theatre_id`;
```

Output:



Q5. Revenue per movie (aggregate functions)

Report for the admin: only confirmed bookings are counted.



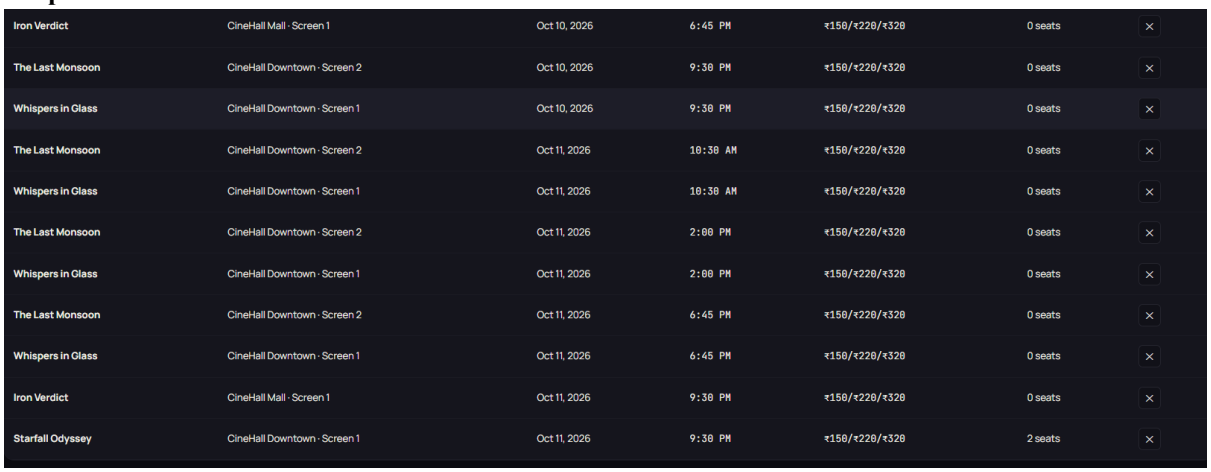
```
const priceOf = { Silver: st.silver_price, Gold:
const total = seats.reduce((sum, s) => sum + Num

let code;
do {
  code = 'CH-' + crypto.randomBytes(4).toString('hex').slice(0, 6).toUpperCase();
} while ((await client.query('SELECT 1 FROM Booking WHERE ticket_code = $1', [code])).rowCount

const b = (await client.query(
  `INSERT INTO Booking (customer_id, showtime_id, ticket_code, total_amount, status)
   VALUES ($1,$2,$3,$4,'Confirmed') RETURNING booking_id`, [req.user.id, showId, code, total]
)).rows[0];
```

@param end
The index to the end of the specified portion of stringOb
including, the character indicated by end. If this value is
stringObj.

Output:



Iron Verdict	CineHall Mall · Screen 1	Oct 10, 2026	6:45 PM	₹150/₹220/₹320	0 seats	×
The Last Monsoon	CineHall Downtown · Screen 2	Oct 10, 2026	9:30 PM	₹150/₹220/₹320	0 seats	×
Whispers in Glass	CineHall Downtown · Screen 1	Oct 10, 2026	9:30 PM	₹150/₹220/₹320	0 seats	×
The Last Monsoon	CineHall Downtown · Screen 2	Oct 11, 2026	10:30 AM	₹150/₹220/₹320	0 seats	×
Whispers in Glass	CineHall Downtown · Screen 1	Oct 11, 2026	10:30 AM	₹150/₹220/₹320	0 seats	×
The Last Monsoon	CineHall Downtown · Screen 2	Oct 11, 2026	2:00 PM	₹150/₹220/₹320	0 seats	×
Whispers in Glass	CineHall Downtown · Screen 1	Oct 11, 2026	2:00 PM	₹150/₹220/₹320	0 seats	×
The Last Monsoon	CineHall Downtown · Screen 2	Oct 11, 2026	6:45 PM	₹150/₹220/₹320	0 seats	×
Whispers in Glass	CineHall Downtown · Screen 1	Oct 11, 2026	6:45 PM	₹150/₹220/₹320	0 seats	×
Iron Verdict	CineHall Mall · Screen 1	Oct 11, 2026	9:30 PM	₹150/₹220/₹320	0 seats	×
Starfall Odyssey	CineHall Downtown · Screen 1	Oct 11, 2026	9:30 PM	₹150/₹220/₹320	2 seats	×

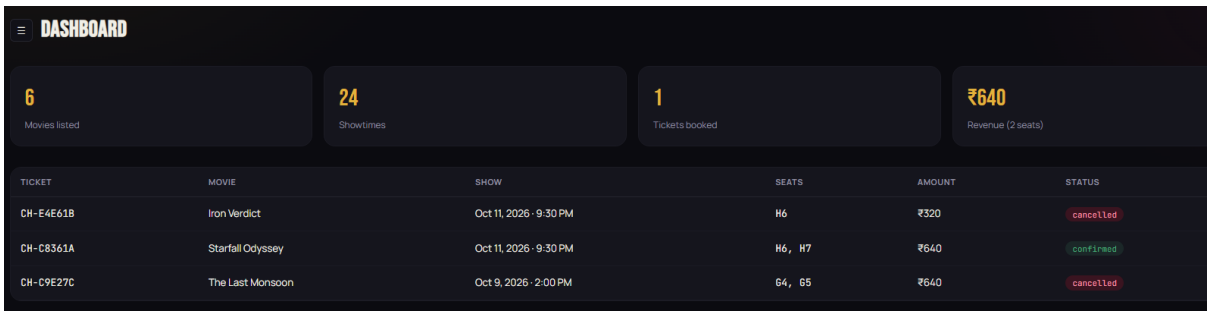
Q6. Cancellation

Shows the history kept when a booking is cancelled.

```
try {
  await client.query('BEGIN');
  const b = (await client.query(
    `SELECT b.booking_id, b.customer_id, b.status, st.show_date < CURRENT_DATE AS past
     FROM Booking b JOIN Showtime st ON st.showtime_id = b.showtime_id
     WHERE b.booking_id = $1 FOR UPDATE OF b`, [req.params.id]
  )).rows[0];
  const isAdmin = req.user.role === 'admin';
  if (!b || (!isAdmin && b.customer_id !== req.user.id)) { await client.query('ROLLBACK'); return
  if (b.status === 'Cancelled') { await client.query('ROLLBACK'); return res.status(400).json({ er
  if (!isAdmin && b.past) { await client.query('ROLLBACK'); return res.status(400).json({ error:

  await client.query(`UPDATE Booking SET status = 'Cancelled' WHERE booking_id = $1`, [b.booking_i
  await client.query(
    `INSERT INTO Cancellation (booking_id, refund_status, reason) VALUES ($1, 'Pending', $2)`,
    [b.booking_id, isAdmin ? 'Cancelled by admin' : 'Cancelled by customer']
```

Output:



The booking row is kept (status Cancelled) and a Cancellation row is added. Because seat availability only counts Confirmed bookings, the seats become free again immediately

Q7. Payments (JOIN)

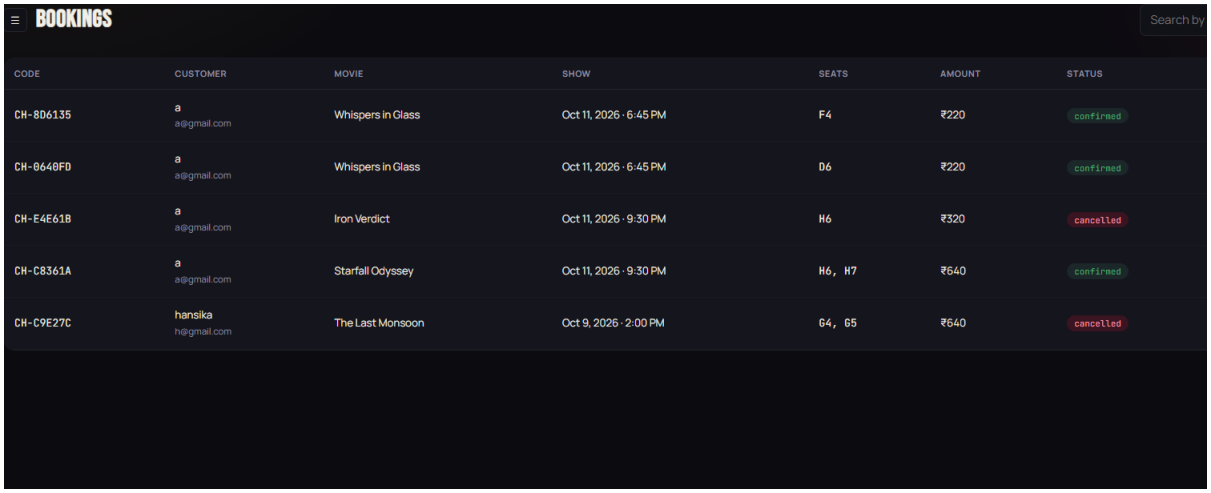
Every booking has a payment record with the chosen method.

```
// Lock this show's row so two people can't grab the same seat at the same moment
const st = (await client.query(
  'SELECT *, show_date >= CURRENT_DATE AS open FROM Showtime WHERE showtime_id = $1 FOR UPDATE', [sl
)).rows[0];
if (!st) { await client.query('ROLLBACK'); return res.status(404).json({ error: 'Show not found' });
if (!st.open) { await client.query('ROLLBACK'); return res.status(400).json({ error: 'This show has

const seats = (await client.query(
  'SELECT seat_id, seat_type FROM Seat WHERE seat_id = ANY($1::int[]) AND screen_id = $2', [ids, st
)).rows;
if (seats.length !== ids.length) { await client.query('ROLLBACK'); return res.status(400).json({ er

const taken = await client.query(
  `SELECT 1 FROM Booking_Seat bs JOIN Booking b ON b.booking_id = bs.booking_id
   WHERE b.showtime_id = $1 AND b.status = 'Confirmed' AND bs.seat_id = ANY($2::int[]) LIMIT 1`, [sl
);
if (taken.rowCount) { await client.query('ROLLBACK'); return res.status(409).json({ error: 'Sorry,
```

Output:



Q8. Booking transaction (preventing double booking)

The booking endpoint runs these SQL steps inside one transaction. The lock in step 1 makes simultaneous bookings for the same show wait for each other, so the seat check in step 3 is always accurate. The total in step 4 is computed from the tier prices stored in the database, never from the browser.

```
const st = (await client.query(
  'SELECT *, show_date >= CURRENT_DATE AS open FROM Showtime WHERE showtime_id = $1 FOR UPDATE', [showId]
)).rows[0];
if (!st) { await client.query('ROLLBACK'); return res.status(404).json({ error: 'Show not found' }); }
if (!st.open) { await client.query('ROLLBACK'); return res.status(400).json({ error: 'This show has already passed' }); }

const seats = (await client.query(
  'SELECT seat_id, seat_type FROM Seat WHERE seat_id = ANY($1::int[]) AND screen_id = $2', [ids, st.screen_id]
)).rows;
if (seats.length !== ids.length) { await client.query('ROLLBACK'); return res.status(400).json({ error: 'Invalid seat selection' });

const taken = await client.query(
  `SELECT 1 FROM Booking_Seat bs JOIN Booking b ON b.booking_id = bs.booking_id
   WHERE b.showtime_id = $1 AND b.status = 'Confirmed' AND bs.seat_id = ANY($2::int[]) LIMIT 1`, [showId, ids]
);
if (taken.rowCount) { await client.query('ROLLBACK'); return res.status(409).json({ error: 'Sorry, one of those seats was just taken.
```

Q.7 Security measures

- Passwords are hashed with scrypt and a random salt before they are stored.
- All SQL uses parameters ($1, $2, ...), which protects against SQL injection.
- Login returns a random token; admin endpoints need an admin token and customer endpoints need a customer token.
- A customer can only read or cancel their own bookings.
- Only the `public/` folder is served, so server code and SQL files are not exposed.
- Text coming from the database is escaped before it is shown in the pages.

8. Testing & Validation

The system was tested by running a script that calls the REST API like the web pages do, and then checks the result in PostgreSQL. Constraint tests insert invalid rows directly into the database. Every test case below was run and the actual output is the real response of the system.

8.1 Functional test cases

ID	Test case	Input	Expected output	Actual output	Result
T1	Customer signup (valid)	name, email, 10-digit phone, password	200 + token + user	200, token issued, user id 1	Pass
T2	Signup with an existing email	same email as T1	409 error message	409 "An account with this email already exists"	Pass
T3	Signup with invalid phone	phone = 12345	400 validation error	400 "Phone must be a 10-digit number"	Pass
T4	Login with wrong password	correct email, wrong password	401 error	401 "Wrong email or password"	Pass
T5	Login with correct credentials	correct email + password	200 + token	200, token issued	Pass
T6	Load seat map for a show	GET /api/shows/1/seats	80 seats, all free	200, 80 seats, 0 booked	Pass
T7	Book 3 seats (A1, A2 Silver; H5 Premium)	showId=1, seatIds=[1,2,75], UPI	200, total = 150+150+320 = 620, ticket code	200, total=620, seats=A1,A2,H5, code=CH-98B211	Pass
T8	Double-booking: book an already booked seat	seatIds=[2] (A2) on the same show	409 seat taken, nothing saved	409 "Sorry, one of those seats was just taken. Please pick again."	Pass
T9	Book more than 8 seats	9 seat ids	400 error	400 "Choose between 1 and 8 seats"	Pass
T10	Book without logging in	no token	401 error	401 "Please sign in again"	Pass
T11	Book a seat id that is not on this screen	seatIds=[300]	400 invalid seat	400 "Invalid seat selection"	Pass
T12	Customer tries an admin action (add movie)	customer token	403 not allowed	403 "Not allowed"	Pass
T13	Admin login with wrong password	admin / wrong	401 error	401 "Wrong admin credentials"	Pass
T14	Admin login (valid)	admin / admin123	200 + token	200, token issued	Pass
T15	Admin adds a movie (INSERT)	valid movie form	200 + new id	200, movie_id=7	Pass
T16	Admin edits the movie (UPDATE)	new title, duration 110	200 success	200, Test Movie 2, 110	Pass
T17	Admin schedules two shows at the same screen, date and time	same screen/date/time twice	2nd request: 409 clash	1st: 200, 2nd: 409 "That screen already has a show at this date and time"	Pass
T18	Admin deletes the movie (DELETE)	movie_id=7	200; row removed	200, rows left: 0	Pass
T19	Customer cancels someone else's booking	other customer token	404 not found	404 "Booking not found"	Pass
T20	Customer cancels own booking	booking_id=1	200; status Cancelled; Cancellation row	200; status=Cancelled; cancellation rows=1	Pass
T21	Cancel the same booking twice	booking_id=1 again	400 already cancelled	400 "Already cancelled"	Pass
T22	Cancelled seats become free again	seat map after T20	0 booked seats	0 booked seats	Pass
T23	Re-book a freed seat	seatIds=[2,3], Debit Card	200, total 300	200, total=300	Pass

8.2 Constraint and data integrity tests

ID	Test case	Input	Expected output	Actual output (PostgreSQL)	Result
T24	DB: duplicate customer email (UNIQUE)	INSERT customer with existing email	unique violation	ERROR: duplicate key value violates unique constraint "customer_email_key"	Pass
T25	DB: invalid booking status (CHECK)	status = 'Pending'	check violation	ERROR: new row for relation "booking" violates check constraint "booking_status_check"	Pass
T26	DB: screen for a non-existent theatre (FOREIGN KEY)	theatre_id = 999	foreign key violation	ERROR: insert or update on table "screen" violates foreign key constraint "screen_theatre_id_fkey"	Pass
T27	DB: duplicate seat on one screen (UNIQUE)	screen 1, row A, seat 1 again	unique violation	ERROR: duplicate key value violates unique constraint "seat_screen_id_seat_row_seat_number_key"	Pass
T28	DB: movie rating above 10 (CHECK)	rating = 11	check violation	ERROR: new row for relation "movie" violates check constraint "movie_rating_check"	Pass
T29	DB: booking without a ticket code (NOT NULL)	ticket_code = NULL	not-null violation	ERROR: null value in column "ticket_code" of relation "booking" violates not-null constraint	Pass

9. Results & Screenshots

The screenshots below show the main screens of the application. They were captured from the interface build; the layout, colours and behaviour are the same in the version connected to PostgreSQL, but the sample movie names in some images differ from the database sample data.

9.1 User interface

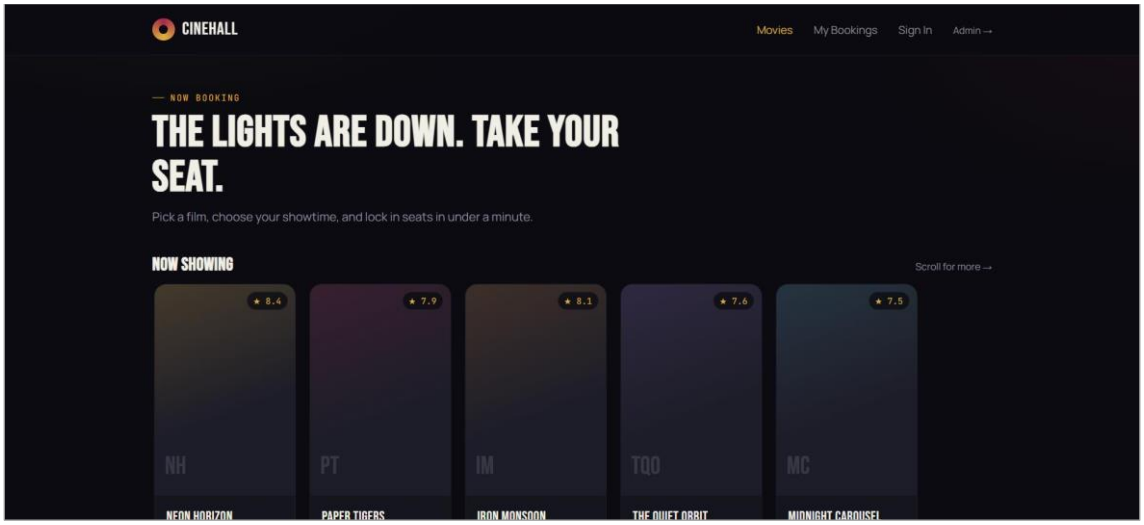


Figure 3: Home page. Now Showing posters with ratings and the navigation bar with Sign In and Admin links.

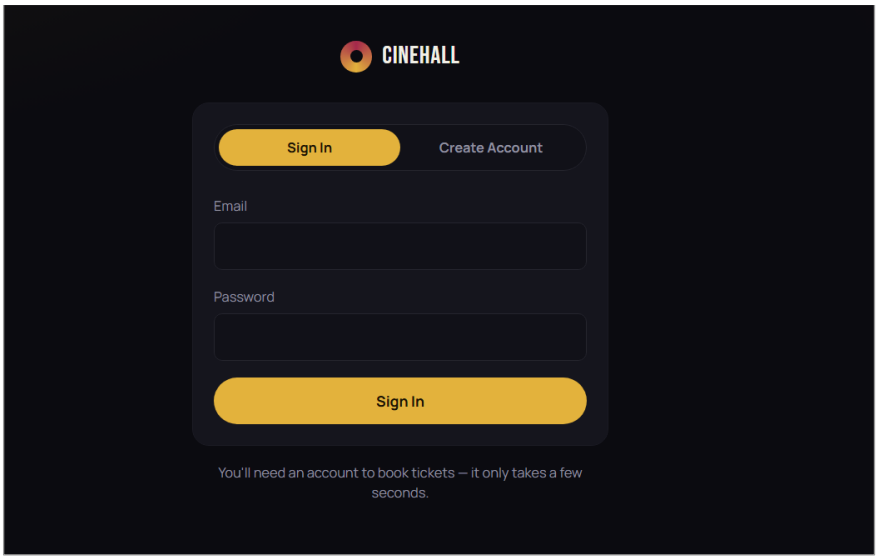


Figure 4: Sign in page. Sign In and Create Account tabs.

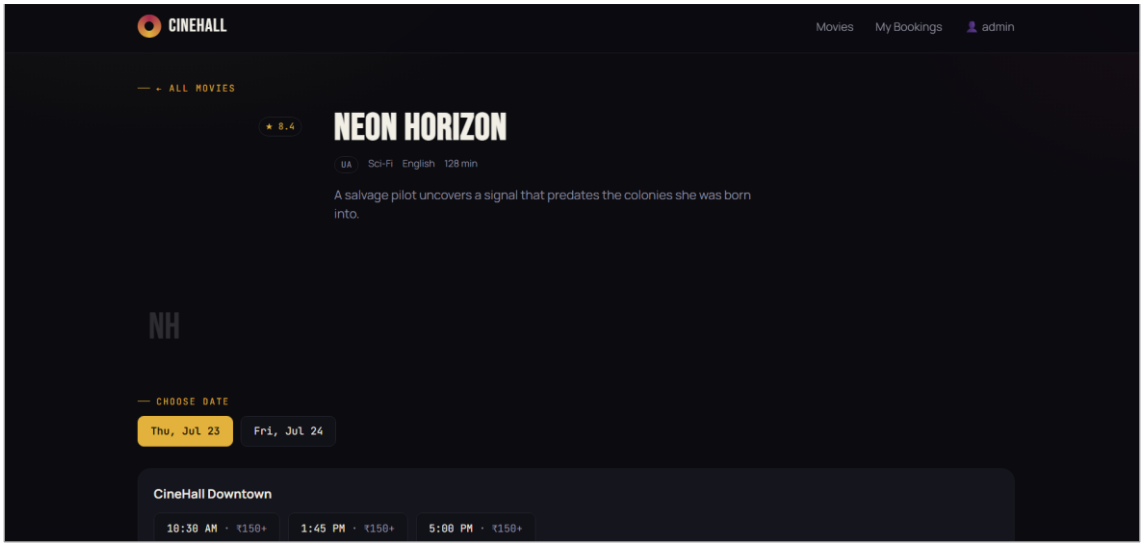


Figure 5: Movie details. Movie information with date chips and cinema cards listing showtimes and starting prices.

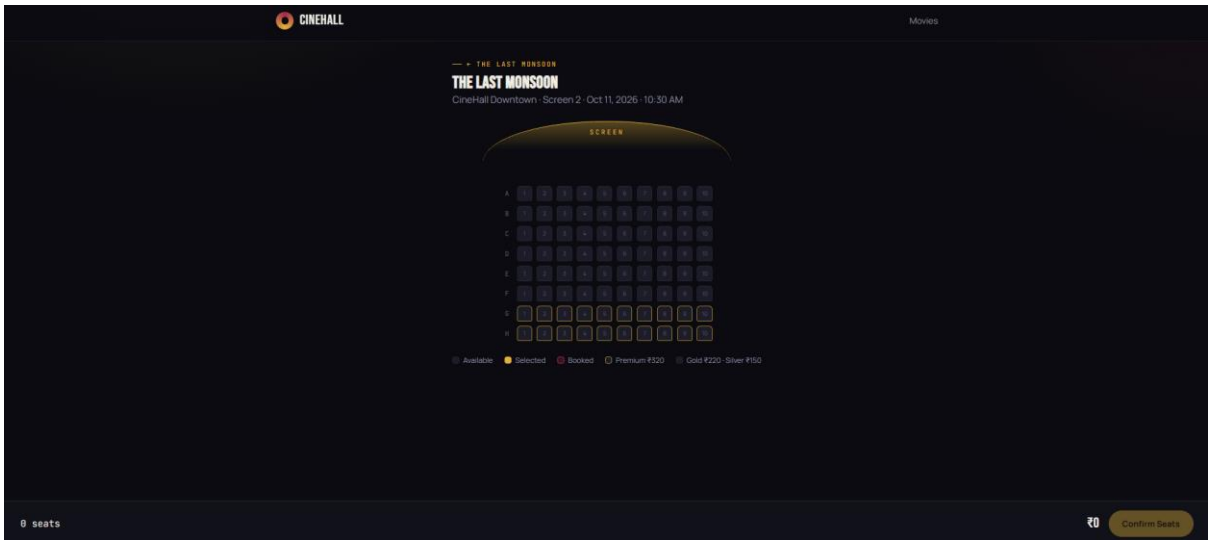


Figure 6: Seat selection. Live seat map with the screen arc, legend and booking bar showing count and total.

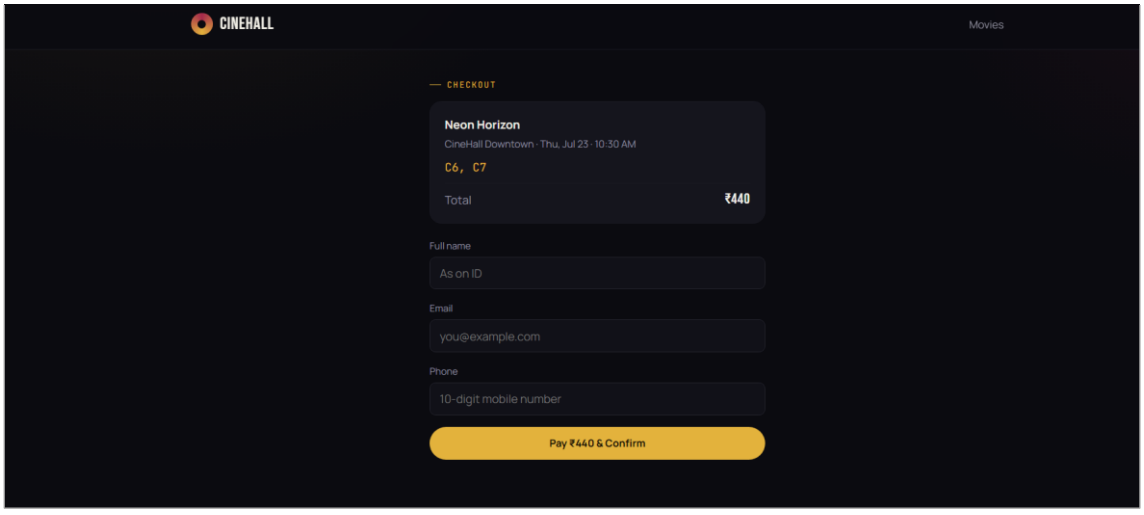


Figure 7: Checkout. Order summary and the confirm-and-pay form.

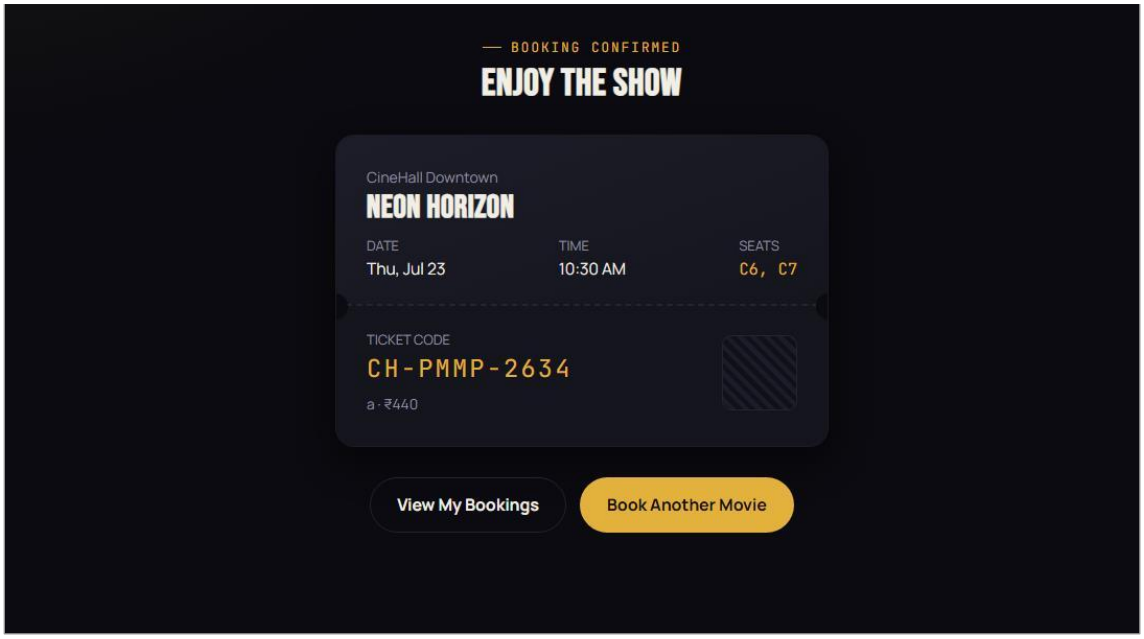


Figure 8: Booking confirmation. The generated ticket with the unique ticket code.

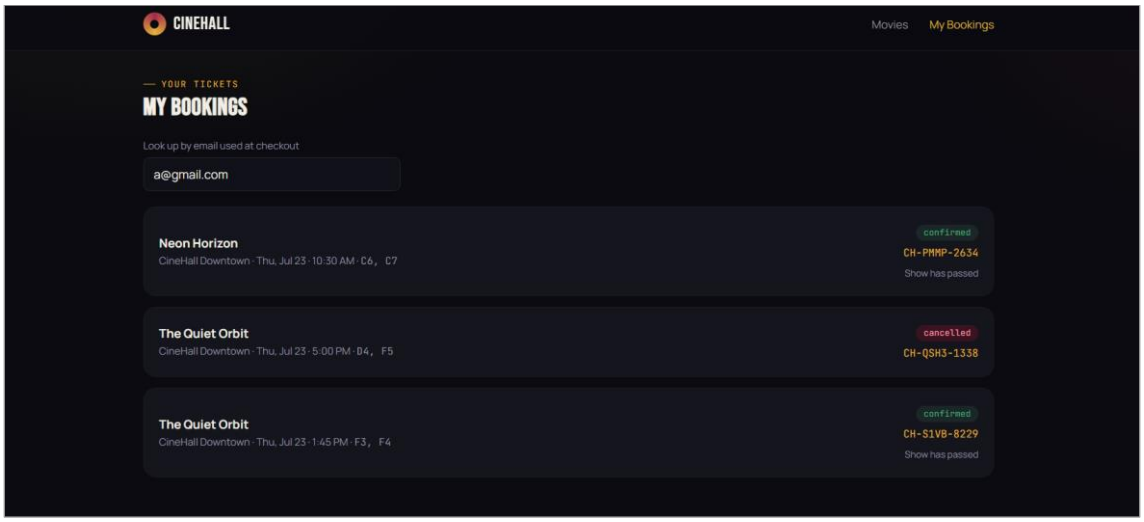


Figure 9: My Bookings. A customer's tickets with status badges; cancelled tickets are shown in red.

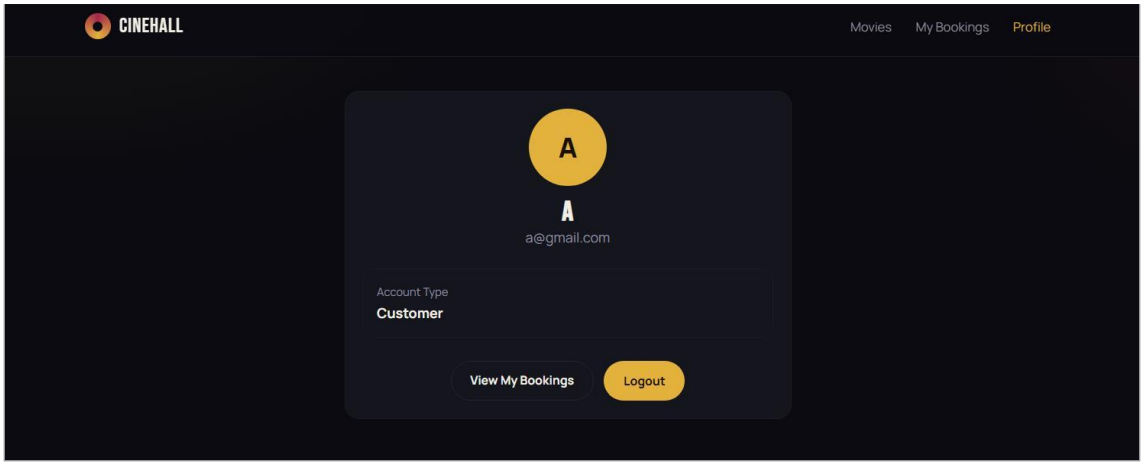


Figure 10: Profile. Customer profile with avatar initials.

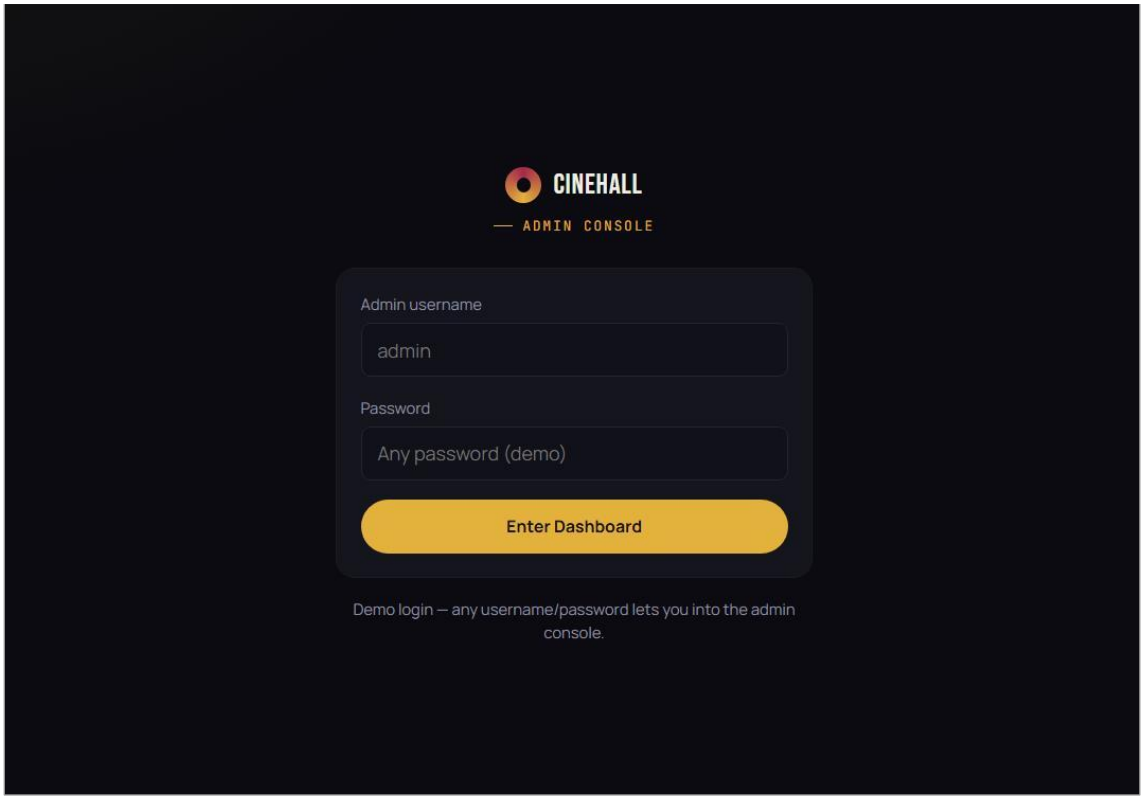


Figure 11: Admin login. Sign-in page of the admin console.

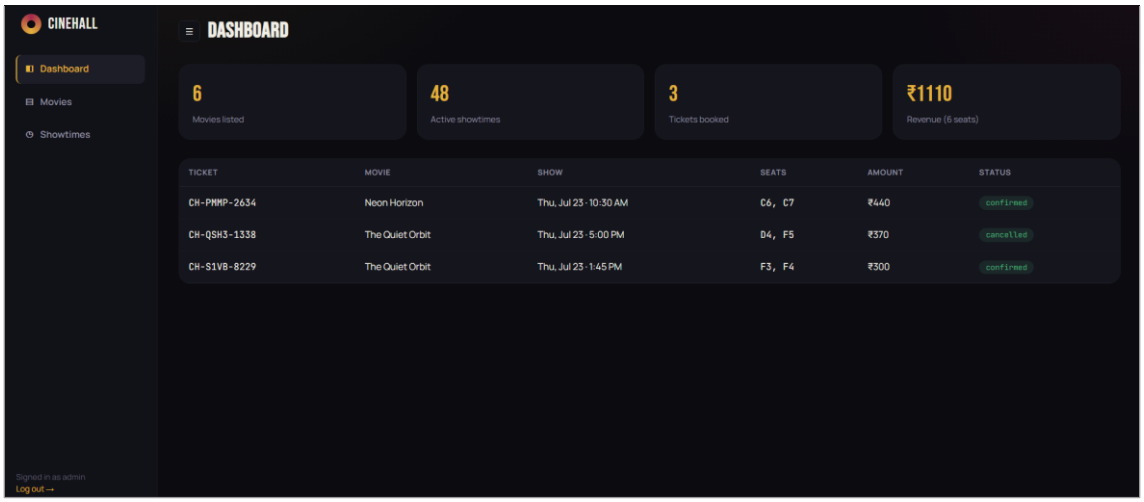


Figure 12: Admin dashboard. Statistics cards (movies, showtimes, tickets, revenue) and the latest bookings.

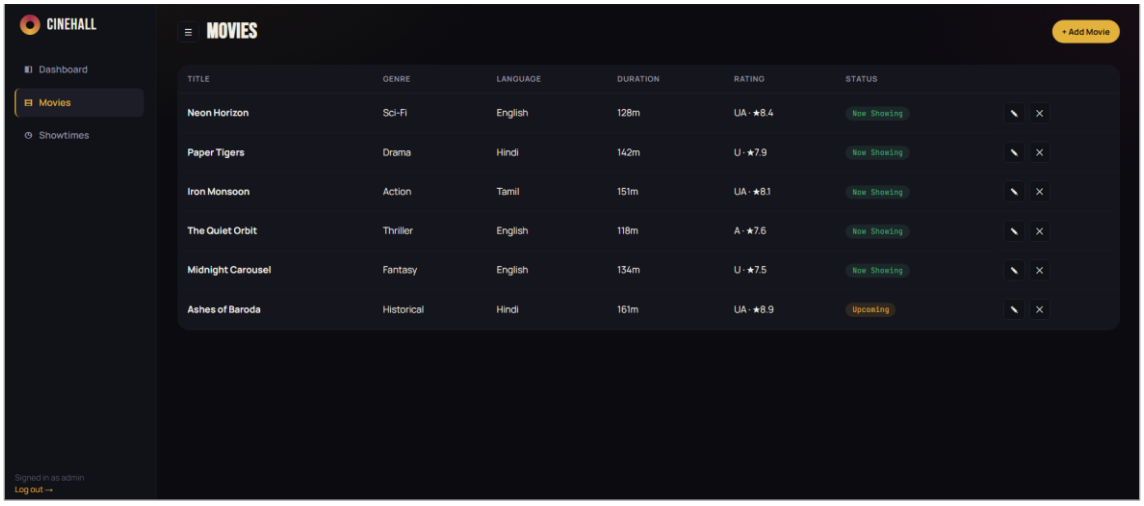


Figure 13: Admin: movie management. Movie table with edit and delete buttons.

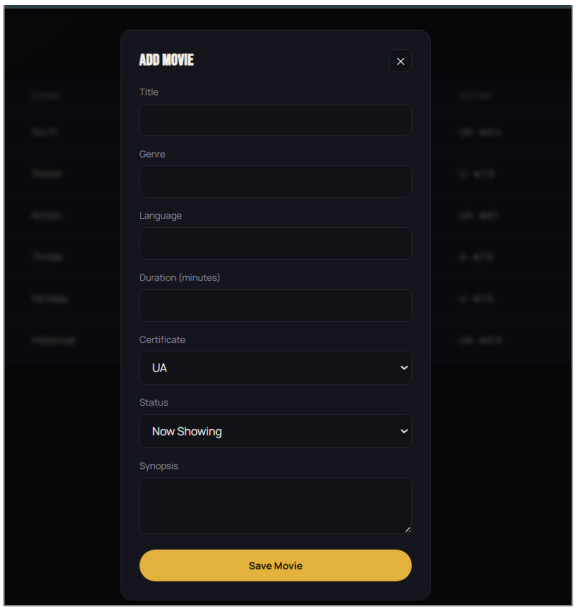


Figure 14: Admin: add movie form. Form used to insert or update a movie.

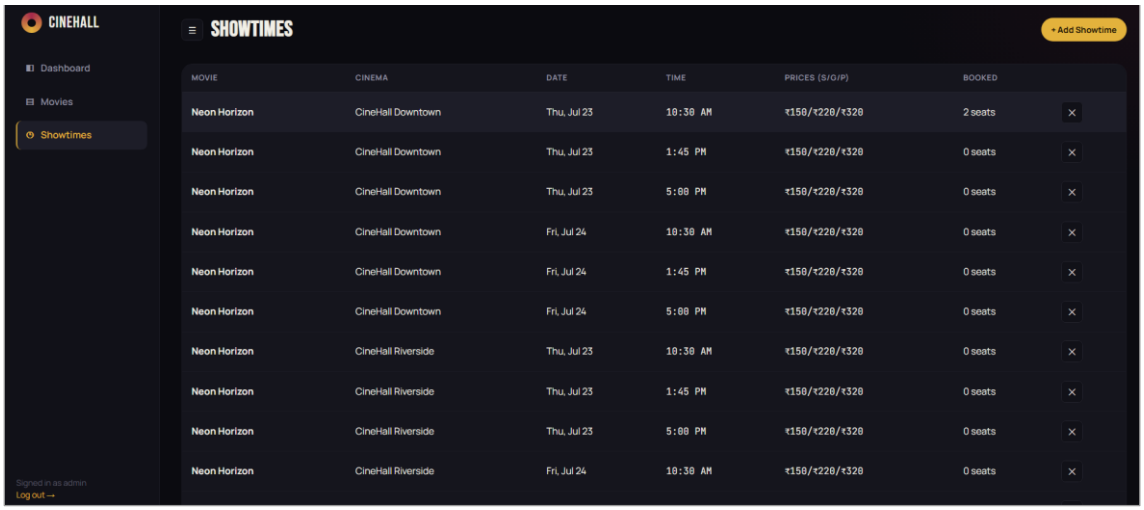


Figure 15: Admin: showtimes. Showtime table with tier prices and number of booked seats.

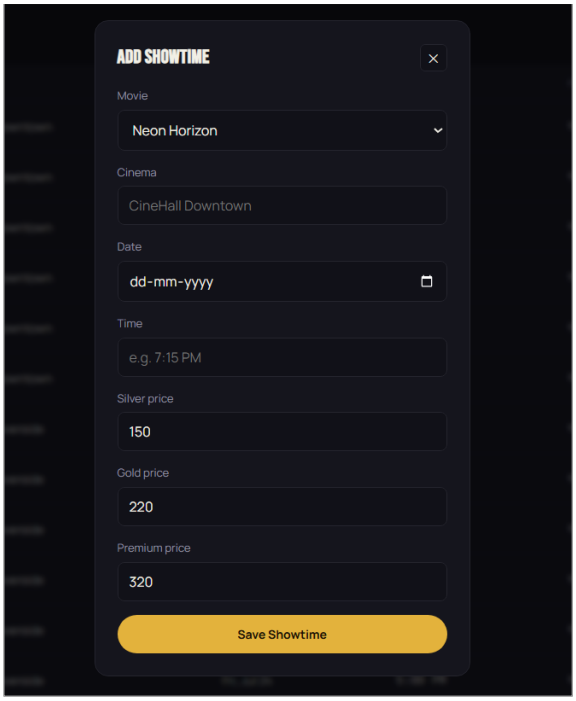


Figure 16: Admin: add showtime form. Form used to schedule a new show.

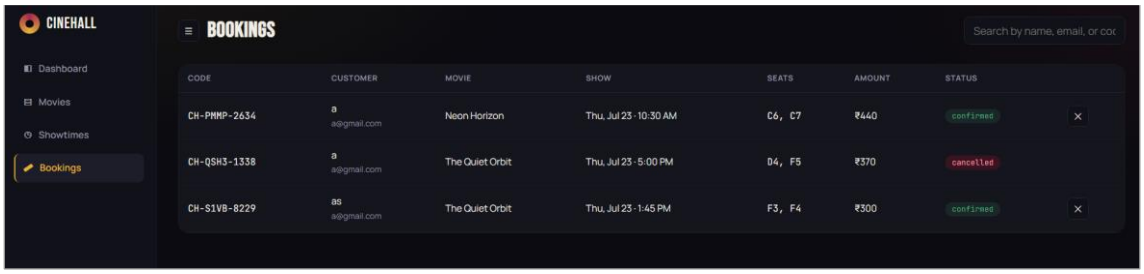


Figure 17: Admin: bookings. Searchable list of all bookings with a cancel button.

10. Challenges Faced & Solutions

Challenge	Solution
Connecting the UI to the database. The first interface kept everything in the browser (localStorage) and its code assumed data was available immediately.	A Node.js/Express API was added. All data functions in `script.js` were rewritten as asynchronous `fetch` calls and every page was updated to wait for the result.
Different names in the UI and the database (for example `lang` and `language`, `rating` and `certificate`, `bookedSeats` and the Booking_Seat table).	The SQL queries rename columns with `AS` aliases, so the API returns exactly the field names the pages expect.
Double booking when two customers choose the same seat at the same time.	Booking runs in a transaction that locks the show row with `FOR UPDATE` and then checks that none of the seats is already confirmed; otherwise it rolls back and tells the user.
Many seats in one booking does not fit in a single column.	The many-to-many relationship was resolved with the junction table Booking_Seat, which also lets seats be released on cancellation.
Cancellation and history. Deleting a booking would lose the record and the sales figures.	A cancelled booking keeps its row with status Cancelled and gets a row in Cancellation; availability and revenue queries only count Confirmed bookings.
Trusting the browser. The first version sent the total price from the page.	The server now reads the seat types and the show prices from the database and calculates the total itself.
Passwords and logins. The demo accepted any password.	Real sign-up and login were added with salted scrypt hashes and random session tokens; admin routes need an admin token.
Dates and times. PostgreSQL DATE values were shifted by one day in JavaScript because of time zones.	Dates are converted in SQL with `to_char` and handled as plain text in the pages.
Table and column names did not match the schema in the first server code, so queries failed.	The server was rewritten against the final schema (Theatre, Customer, Booking_Seat and so on) and tested end to end.
Exposed project files. Serving the whole project folder would expose the server code.	Only the `public/` folder is served.

11. Conclusion & Future Enhancements

11.1 Conclusion

CineHall shows how a well-designed relational database supports a real application. The ten-table schema is normalized to BCNF and protected by keys and constraints. The web interface and the Express API connect to it so that customers can browse movies, choose seats, book and cancel, and administrators can manage movies and showtimes and watch revenue. Transactions and row locking prevent double booking, cancellations are kept for history, and the server calculates prices and checks permissions. The 29 test cases, which all passed, confirm that the objectives of the project were achieved.

11.2 Future enhancements

- **Payment deadline:** keep new bookings Pending for 10 minutes and release the seats automatically if payment is not completed (a scheduled job or trigger).
- **Full overlap check:** prevent overlapping shows on a screen using movie duration plus a cleaning buffer.
- **Cancellation rule and refunds:** allow cancellation only up to 2 hours before the show and process refunds through a payment gateway.
- **More roles:** separate Theatre Admin and System Admin accounts with their own permissions.
- **Real payments:** integrate a payment gateway such as UPI or card providers.
- **Tickets:** e-mail or SMS tickets with a scannable QR code.
- **Analytics:** views and reports for revenue by day, occupancy rate and popular movies or genres.
- **Dynamic pricing:** a separate Price table by day, time and seat type, as proposed in the abstract.
- **Security and deployment:** HTTPS, token expiry stored in the database, and cloud hosting with managed PostgreSQL.
- **Mobile app and movie posters:** a mobile front end and image support for posters and trailers.

References

[1] R. Elmasri and S. B. Navathe, Fundamentals of Database Systems, 7th ed., Pearson, 2016.

[2] A. Silberschatz, H. F. Korth and S. Sudarshan, Database System Concepts, 7th ed., McGraw-Hill, 2019.

[3] PostgreSQL Global Development Group, PostgreSQL Documentation (SQL commands, constraints, transactions and explicit locking). https://www.postgresql.org/docs/

[4] node-postgres (`pg`) documentation. https://node-postgres.com/

[5] Express.js documentation. https://expressjs.com/

[6] Node.js documentation. https://nodejs.org/en/docs

[7] MDN Web Docs: HTML, CSS, JavaScript and the Fetch API. https://developer.mozilla.org/

[8] pgAdmin documentation (ERD tool and query tool). https://www.pgadmin.org/docs/

[9] Claude (Anthropic), AI assistant used to generate the first version of the user interface and to help connect it to the database.

[10] CineHall source code repository. https://github.com/Hansi-111/movie-ticket-booking-system-ui

[11] Course material, 23CSE202 Database Management Systems, Amrita School of Computing, Amritapuri Campus.